Optical Flow Based Video Classification with Visual Attention using Recurrent Convolutional Neural Network

Xinran Li* Anwesan Pal* Ahmed Qureshi* Carlos Nieto-Granda* Manuela Vasconcelos
Department of Electrical and Computer Engineering, Jacobs School of Engineering
University of California San Diego

***Abstract*— One of the most curious facets of the human visual system is the presence of attention. Rather than compressing an entire image into a static representation, attention allows for salient features to dynamically come to the forefront as needed. This is especially important when there is clutter in an image or a perceivable agents behavior is desired. Perceivable behaviors of an agent are crucial for many real-world applications. In this paper, we focus on classification with visual attention. We use Recurrent Convolutional Neural Networks (R-CNN) and Recurrent Attentive Convolutional Neural Networks (R-ACNN) to focus on certain parts of an input image instead of processing it entirely at a fine scale. Experiments using real camera data validate the arguments.**

***Index Terms*— Optical Flow, Visual Attention, Convolutional Neural Networks, Machine Learning.**

I. Introduction

The study of attention has classically focused on the physical characteristics that determine whether stimuli are attended. William James [1] defined attention as taking possession of one out of several simultaneously possible objects or train of thoughts by the mind. The modern study of attention continues to work within this broad definition. It is generally accepted that an individual is only aware of a small fraction of the information provided to the brain by the sensory systems.

The computer vision models of attention perform remarkably well in most of the applications where static images or images from a video stream are manually fed to the model in order to identify the most salient/task-relevant stimuli. In case of some real-time applications where the current visual input of the attention model is to be determined by the decision output of the model (i.e., the focus of attention) at the immediate past, the traditional computer vision models of visual attention faces severe limitations in a number of aspects [2], [3] and [4]. Using a visual attention model as a component of robotic cognition is an example of such applications. In this case, the attention model should be able to locate the behaviorally relevant stimuli in an ongoing stream of visual input and respond to it, perform learning in an on-line fashion and with minimal human supervision, and apply the learned knowledge for guiding the attention behavior in arbitrary environmental settings.

The paper is organized as follows: In section II, we provide a brief background of optical flow and Neural Network literature which has been implemented thus far. A presentation of our technical approach is given in Section III. This section includes the development of our visual attention framework, our proposed neural network model for video classification. Section IV includes a detailed description of the data collection and pre-processing, and the model architecture and hyper-parameters of our implementation. The main technical points and results are summarized in Section V. Finally, future work will be discussed in Section VI.

*All these authors contributed equally to the work.

II. Related Work

The study of Optical Flow in Computer Vision has long been an interesting domain, right from its inception. [5], [6] [7], and [8] give a background on what Optical Flow is and how it can be determined in a computer vision problem. In the Approach section of this report, we further give its application to our report. Convolutional Neural Networks are the primary form of architecture used in most Computer Vision problems these days. Deep Learning essentially involves having a series of convolutional layers, followed by multiple fully-connected layers which enable the step-wise detection of features of importance, finally leading to the higher level-linguistics representations [9].

Gathering temporal information is of prime importance for video-based classification techniques. [10] uses this and introduces the concepts of Early Fusion and Late Fusion. Early Fusion combines information over a time window at a pixel-level, while Late Fusion places two separate single-frame networks until the last convolution layer with frames being separated periodically. Another representation of fusion is the Two-Stream Convolutional Networks [11] used for classifying videos, where the spatial and temporal information form the two steams and are processed separately. Recurrent Neural Networks are useful when we associate a memory with a sequence of data gathered across several frames in a video [12].

Our model uses the above-mentioned concepts and expands them to incorporate attention in a series of frames to classify videos. We tested our model on two labeled datasets containing human actions over a length of video. The larger of the two is a custom-made dataset which involves some common human actions performed and captured in a short video format.

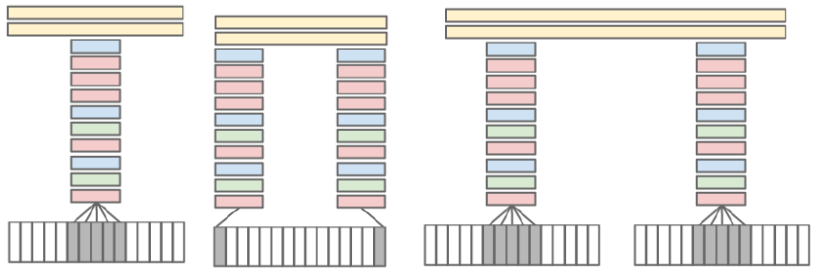

Fig. 1: From left to right: Comparison of early-fusion, late-fusion, and two-stream methods.

III. Methodology

A. Estimating Camera Motion and Scene Structure Between Video Frames

To show the connection between camera motion and image point motion, we first introduce a general optical flow equation:

$$u(x) = N_x(d\tau + \Omega x), \tag{1}$$

where $x \in \mathbb{R}^3$ is a point on the imaging surface, $\hat{x} = \|x\|^{-1}x$ is this vector normalized, $u(x) \in \mathbb{R}^3$ is the optical flow vector at that point (always tangent to the imaging surface), d is the inverse distance from the camera principal point to the scene point imaged at x, τ is the camera translation vector, and W is the camera angular velocity. N_x is the linear transformation, a 3×3 matrix, that projects flow on the unit sphere at x to flow on the true imaging surface.

1) Lucas-Kanade Method: We calculate the motion of each person via Lucas-Kanade Method [6]. This method assumes the small and constant velocity between two nearby frames. We want to minimize the sum of squared error between template T (or the first frame and the second frame I warped back onto the template. We setup the cost function as:

$$\sum_{x,y}[I(W(x,y;p)) - T(x,y)]^2 \tag{2}$$

We minimize the equation as a non-linear optimization problem because pixel values $I(x,y)$ are independent to coordinates x, y. Therefore, we optimize the method by descent step Δp:

$$\sum_{x,y}[I(W(x,y;p) + \Delta p) + T(x,y)]^2 \tag{3}$$

We update the step with

$$p_{t+\Delta} = p_t + \Delta p \tag{4}$$

Next we applied the first order term of Taylor series

$$\sum_{x,y}[I(W(x,y;p)) + \nabla I \frac{\partial w}{\partial p}\Delta p - T(x,y)]^2 \tag{5}$$

We assume the motion is pure translation.

$$W((x,y;p) + \Delta p)) = \begin{vmatrix} x + p1 \\ y + p2 \end{vmatrix} \tag{6}$$

$$\frac{\partial w}{\partial p} = \begin{vmatrix} 1 & 0 \\ 0 & 1 \end{vmatrix} \tag{7}$$

Then, we solve for Δp by Gauss-Newton Method where Δp is

$$\Delta p = H^{-1}\sum_{x,y}[\nabla I]^T[T(x,y) - I(W(x,y;p))] \tag{8}$$

and the Hessian is

$$H = \sum_{x,y}[\frac{\partial w}{\partial p}]^T[\nabla I \frac{\partial W}{\partial p}] \tag{9}$$

$$H = \sum x, y[\nabla I]^T[\nabla I] \tag{10}$$

For a clearer visual analysis, Figure 2 shows the steps to compute optical flow from a sequence of two pictures. The arrows represent the motion which the object has moved with respect to the camera view.

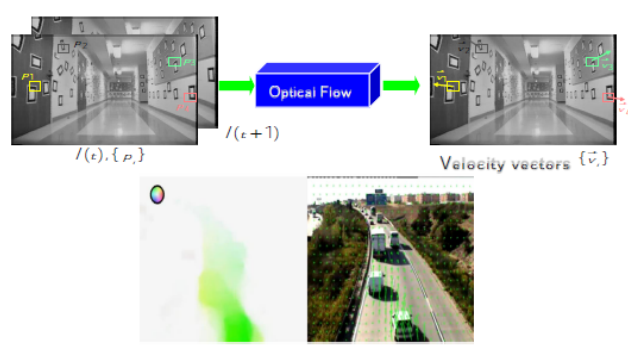

Fig. 2: Diagram of computing optical Flow via Lucas-Kanade Method. In the above image, optical flow gives the velocity gradients of the pixels which move in position over time. In the lower image, we highlight the portion of the image which is in motion (the vehicles).

B. Neural Networks

In this section, we describe our proposed neural network model for video classification. We present two different models i.e., Recurrent Convolutional Neural Networks (R-CNN: see Fig. 3) and Recurrent Attentive Convolutional Neural Networks (R-ACNN: see Fig. 4). R-ACNN extends R-CNN by adding the neural attention module. Attention model enables a neural network to focus on certain parts of an input image instead of processing it entirely at a fine scale. This region selection reduces the number of training parameters as well as the computational operations. Beside computational benefits, attention models also provides information about where a neural network is looking at, while taking any decision. Rest of the section is divided up according to the different neural models of R-CNN and R-ACNN i.e., 1) Convnets; 2) Long Short-term Memory (LSTM) network; and 3) Attention network (G) (used by R-ACNN). The flow of information between the different modules of R-CNN and R-ACNN can be seen in Fig. 3 and Fig. 4.

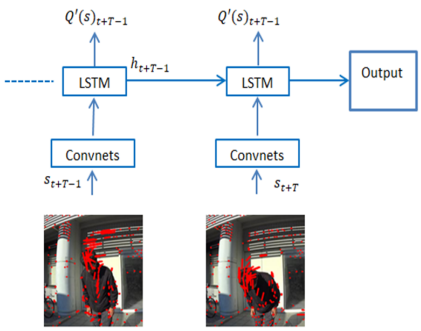

Fig. 3: Recurrent Convolutional Neural Network

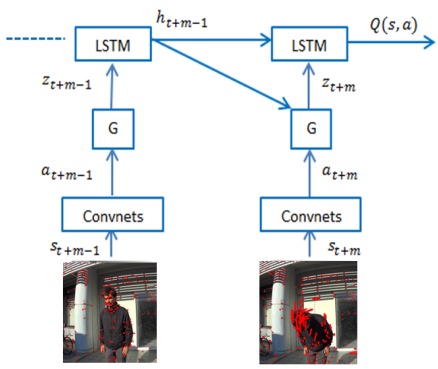

Fig. 4: Recurrent Attentive Convolutional Neural Network

1) ConvNets: The ConvNets take preprocessed visual frame as input in each time-step and transforms it into L feature vectors, each of which provides D-dimensional representation of a part of an input image i.e, $a_t = \{\mathbf{a}_t^1, \cdots, \mathbf{a}_t^L\}, \mathbf{a}_t^l \in \mathbb{R}^D$. This feature vector is either taken as an input by the linear layer in case of R-CNN or by attention network in case of R-ACNN to generate an annotation vector $z \in \mathbb{R}^D$.

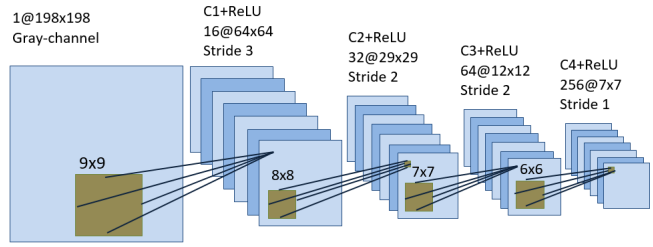

Fig. 5: The structure of the Convolutional network encoder.

2) LSTM: We employ the following implementation of LSTM network:

$$\begin{pmatrix} i_t \\ f_t \\ o_t \\ g_t \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} M \begin{pmatrix} h_{t-1} \\ z_t \end{pmatrix} \tag{11}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \tag{12}$$

$$h_t = o_t \odot \tanh(c_t) \tag{13}$$

where i_t, f_t, o_t, c_t, and h_t correspond to the input, forget, output, memory and hidden state of the LSTM respectively. Let d be the dimensionality of all LSTM states and matrix $M : \mathbb{R}^a \rightarrow \mathbb{R}^b$, in equation 3, is an affine transformation of trainable parameters with dimension $a = d + D$ and $b = 4d$. As shown in equation 3-5, the LSTM network takes the annotation vector $z_t \in \mathbb{R}^D$, previous hidden state h_{t-1}, and the previous memory state c_{t-1} as an input in order to produce the next hidden state h_t. This hidden state h_t is given to the attention network G (in case of R-ACNN) and to the linear output layer for generating the annotation vector z_{t+1} at next time step and for providing the output prediction value for each of the legal classes, respectively.

3) Attention network: The attention network (for R-ACNN) generates a dynamic representation, called annotation vector z_t, of the corresponding parts of an input image at time t. The attention mechanism ϕ, a multilayer perceptron, takes a D-dimensional L feature vectors a_t and a previous hidden state h_{t-1} of the LSTM network as an input for computing the positive weights β_t^l for each location l. The weights β_t^l are computed as follow:

$$\beta_t^l = \frac{\exp(\alpha_t^l)}{\sum_{k=1}^{L} \exp(\alpha_t^k)}, \tag{14}$$

where

$$\alpha_t^l = \phi(a_t^l, h_{t-1}), \tag{15}$$

and

$$\beta_t^l = \frac{\exp(\alpha_t^l)}{\sum_{k=1}^{L} \exp(\alpha_t^k)}. \tag{16}$$

The annotation vector z_t is given by

$$z_t = \sum_{l=1}^{L} \beta_t^l a_t^l. \tag{17}$$

This annotation vector is used by LSTM for computing next hidden state.
There are two types of attention networks [13] in the literature: the soft and hard attention network. The attention network used in our method is the soft attention network and unlike hard attention network, it is fully differentiable and deterministic.

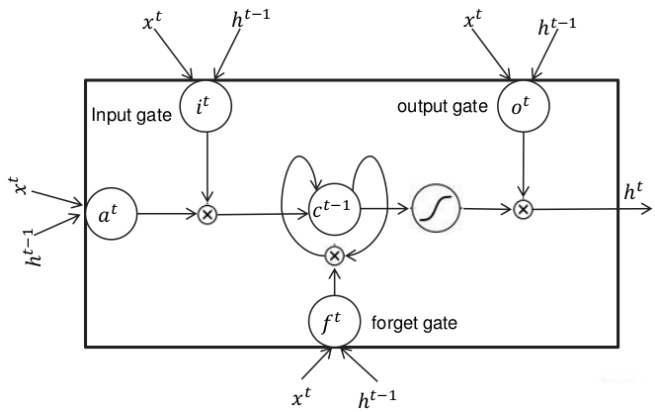

Fig. 6: The structure of a long short-term memory (LSTM) network.

IV. Technical Approach

This section outlines the implementation details of our project. The algorithms were implemented in Pytorch. The system used for training our model has 3.40GHz octa-core Intel Core i7 processor with 32 GB RAM and nVidia GeForce GTX 1080 GPU. The rest of the section explains various modules of the project.

A. Data Collection and Pre-processing

The dataset we collected was more elaborate and it comprised of 5 classes that we use in daily life communication. Each gesture is depicted in Figure 7, along with a representative caption. Moreover, because we wanted to ensure that the network is invariant to irrelevant factors such as background or lighting and sensitive only with respect to gesture, we collected videos in 7 different locations, outdoors and indoors. Furthermore, participants were recorded with three cameras, positioned at different angles and heights. As the gestures involve motion to be clearly distinguishable, we decided to include the temporal information by giving the network a series of consecutive frames, instead of a single image. From each gesture clip, we randomly sampled 5 frames from 4 consecutive frame intervals. Around 160 samples were obtained per gesture class. Afterwards the images are rescaled and cropped in the center area such that their dimensions decrease to 210×210. After being transformed to YUV input, frames go through the process of computing optical flow [5]. Finally, the images were globally normalized, such that the mean of all pixel values was 0 and the standard deviation 1.

B. Model Architecture and hyper-parameters

This section provides the details of the architecture of our R-CNN and R-ACNN model. The convolutional neural network consists of four convolution layers each of which is followed by a non-linear rectifier function. The input dimension to the CNN is $2 \times 210 \times 210$. The convolution layer 1, 2, 3 and 4 convolves 16 filters of 9×9, 32 filters of 8×8, 64 filters of 7×7 and 256 filter of 6×6, respectively. The stride of the convolution layers 1, 2, 3 and 4 are 3, 2, 2 and 1 respectively. The CNN outputs 256 feature maps of dimension 7×7. The output feature maps from CNN are given to the attention network which takes 49 vectors each of size 256. To be consistent with attention network, the LSTM network also has 256 units. To generate prediction-values for the five classes, the output of the LSTM is transformed to five units through a linear layer preceded by non-linear rectifier unit. Finally, the cost function is negative log-likelihood and the learning rate was kept at 10^{-3}.

V. Results and Discussion

The dataset consists of 780 videos in total, with 624 (or 80%) for training and 156 (or 20%) for testing. Five models, including Early Fusion, Late Fusion, R-CNN, Two-stream, and R-ACNN, have been implemented. Where applicable, optical flow have been added to the dataset. The results of these experiments are shown in Table I.

TABLE I: The accuracy for each of the five models, with and without optical flow.

Models	Grayscale Accuracy	Optical Flow Accuracy
Early Fusion	42.8%	–
Late Fusion	57.3%	86.7%
R-CNN	59.0%	**98.2%**
Two Stream	87.3%	87.3%
R-ACNN	57.8%	**97.4%**

The results suggest that the best models are our R-CNN and R-ACNN with optical flow, having accuracy above 97%. The next best models, two-stream and late fusion, have only been able to achieve 87% accuracy. Other methods, especially the ones that do not use optical flow, could not achieve an accuracy higher than 60%.

We observe that using optical flow significantly boosts accuracy in late fusion, R-CNN and R-ACNN models. The two-stream model does not appear to be able to effectively use the optical flow. In contrast, the R-CNN and R-ACNN models get the most significant boost in accuracy by using optical flow. Curiously, R-ACNN does not perform better than R-CNN in our experiments. In fact, R-ACNN has slightly lower accuracy than R-CNN, regardless of the use of optical flow. We believe this difference in accuracy is due to the fact that attentive model focuses only on a small region of the input, where as the non-attentive model uses the entire input. If the attention is in the wrong region, then the classification becomes much less likely to be correct.

Fig. 7: Examples of input sequences for each class. From top to bottom, the classes are “bye”, ”don’t know”, “konnichiwa”, “quiet”, “high five”.

Fig. 8: The attention masks generated from training the recurrent attentive convolutional neural network (R-ACNN). The regions with higher attention are marked with brighter color.

Fig. 9: The attention masks for two training examples at epochs 0, 2, 5, 20, and 65 demonstrate the convergence of attention masks to a local region.

Fig. 10: In some cases, the attention masks do not converge or converge to wrong locations.

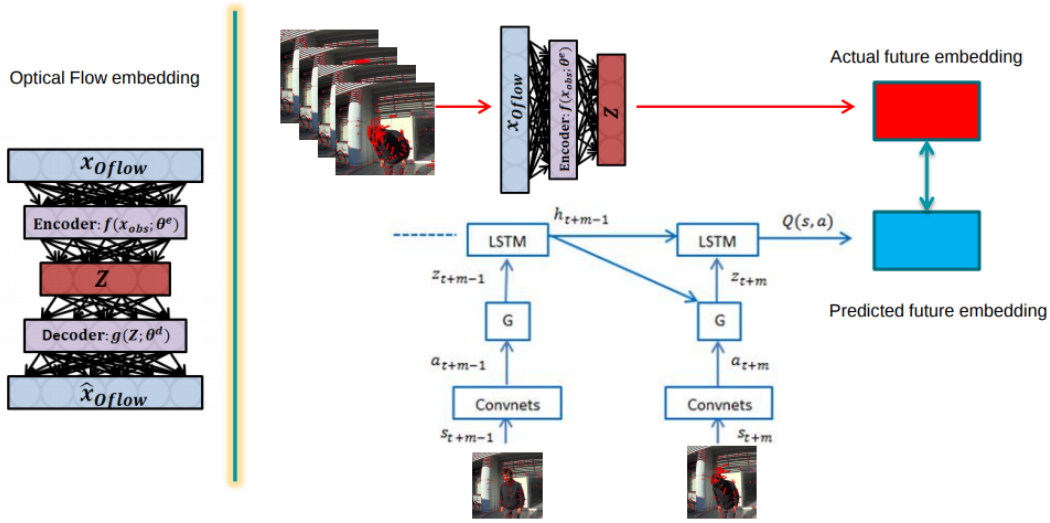

Fig. 11: Unsupervised optical flow prediction from sequences.

What, then, is the advantage of using an attentive model such as R-ACNN over R-CNN? An answer to this question is that the attention masks which the attentive model produces may provide intuitive ways to interpret the trained models. Figure 8 shows the attention masks generated by the R-ACNN. The attention masks are produced by performing a convolution of a Gaussian blur filter with the annotation vector z_t, which is reshaped and rescaled to match the input frame size, and then adding it on top of the input image with a non-zero alpha channel.

We observe that in all examples except for "shh", the attention masks focus on "high-activity regions", ie the regions where significant motion is present. For the "konnichiwa" action, the region is the upper body; for the "don't know", "wave" and "high five" actions, the elbow. For the "shh" action, the model identifies the region of interest outside the body. We believe that these highlighted regions demonstrate the regions where the model finds the most "telling" about the action in question, similar to PCA identifying the most significant factors of a model. The attention masks are especially useful for video and image data, since they allow even non-technical persons to peek into the network's mechanism. This might be important to consider in designing service robots, as users may be more willing to interact with a robot that displays its attention.

How are the attention masks formed by training? Figure 9 shows the evolution of two attention masks over training epochs. In both examples, the attention masks converge to a local region. Before training, the attention is dispersed across large areas in the images. With training, the attention gradually collapses into a few prominent regions (usually two) before one region eventually dominates. The examples suggest that the model is the healthiest around epoch 20, as the model at this point has formed a few regions of interest while not yet assigning all trust to a single region. This might prevent over-fitting, to which the relatively small size of our dataset is prone.

VI. Future Work

We plan to expand our work in several ways.

First, the classification architecture can be extended to perform label classification on future frames. In other words, the network aims to predict the labels of future frames. In this architecture, the output labels are simply shifted a fixed amount forward in time.

Second, an encoder for the feature maps (or embedding) from the optical flow can be obtained by training an auto-encoder for the annotation vector z_t. The auto-encoder consists of an encoder and a decoder, which are symmetrical with respect to the annotation vector.

The prediction and encoder networks can be combined to form a single network that performs unsupervised optical flow prediction on image sequences. Figure 11 illustrates one possible structure of such type of networks. The future embedding produced by the prediction network is compared to the actual future embedding produced by the encoder, and the error is back-propagated into the network to allow unsupervised training.

Such type of networks may find its use in, for example, predicting human actions from videos, which can be useful for a interactive robot or surveillance system, while allowing training on very large video datasets that does not contain annotation or labeling.

References

[1] W. James, *The principles of Phsychology*. Dover Publications, 1950.

[2] M. Begum, G. K. I. Mann, and R. G. Gosine, "A biologically inspired bayesian model of visual attention for humanoid robots," in *2006 6th IEEE-RAS International Conference on Humanoid Robots*, 2006, pp. 587–592.

[3] M. Begum, G. K. I. Mann, R. G. Gosine, and F. Karray, "Object- and space-based visual attention: An integrated framework for autonomous robots," *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 301–306, 2008.

[4] M. Begum, F. Karray, G. K. I. Mann, and R. G. Gosine, "Re-mapping of visual saliency in overt attention: A particle filter approach for robotic systems," in *2008 IEEE International Conference on Robotics and Biomimetics*, 2009, pp. 425–430.

[5] B. K. Horn and B. G. Schunck, "Determining optical flow," *Artificial intelligence*, vol. 17, no. 1-3, pp. 185–203, 1981.

[6] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proceedings of the 7th International Joint Conference on Artificial Intelligence - Volume 2*, ser. IJCAI'81, 1981, pp. 674–679.

[7] D. J. Fleet and Y. Weiss, "Optical flow estimation," in *Handbook of Mathematical Models in Computer Vision*, 2006.

[8] S. Sun, Z. Kuang, W. Ouyang, L. Sheng, and W. Zhang, "Optical flow guided feature: A fast and robust motion representation for video action recognition," *CoRR*, 2017.

[9] "CS224d: Deep Learning for Natural Language Processing, howpublished = http://cs224d.stanford.edu/, note = Accessed: 3-18-2017."

[10] A. Karpathy, G. Toderici, S. Shetty, T. Leung, R. Sukthankar, and L. Fei-Fei, "Large-scale video classification with convolutional neural networks," in *CVPR*, 2014.

[11] K. Simonyan and A. Zisserman, "Two-stream convolutional networks for action recognition in videos," in *Advances in Neural Information Processing Systems*, 2014.

[12] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, http://www.deeplearningbook.org.

[13] K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhudinov, R. Zemel, and Y. Bengio, "Show, attend and tell: Neural image caption generation with visual attention," in *International Conference on Machine Learning*, 2015, pp. 2048–2057.